

ML - Assignment 4

Christian Parker

November 2023

1.
 - Logistic Regression vs Neural Network
 - Bias

Since it's a linear model, Logistic Regression is prone to underfit higher complexity models and therefore is prone to high bias. In contrast, Neural Networks can track higher complexity models and will have lower bias.
 - Variance

The complexity of Neural Networks makes them prone to overfitting which means higher variance. Logistic regression is unlikely to have this variance as it will underfit most datasets.
 - Logistic Regression vs Decision Tree
 - Bias

Logistic Regression has higher bias because it can't fit the training data as well as a decision tree can. A straight line can't divide the data up as well as a decision tree can.
 - Variance

Logistic Regression has lower variance than Decision Trees because Decision Trees can slice the data up into more complex patterns which makes them prone to overfitting and higher variance.
 - Decision Tree vs Random Forest
 - Bias

Random forest is just the decision tree algorithm with bagging. Bias should be similar across both because bagging doesn't affect bias much.
 - Variance

Random forest will have lower variance because bagging lowers variance.
 - Decision Tree vs Gradient Boosted Tree
 - Bias

Gradient Boosted Tree is just the decision tree algorithm with boosting. Gradient Boosted Tree will have lower bias because boosting decreases bias.

- Variance
Boosting won't affect variance much.
 - Logistic Regression vs 1NN classifier
 - Bias
1NN tends to have slightly lower bias because it can fit more complex models.
 - Variance
1NN will have high variance because it will match the training set perfectly but the testing set results will be highly influenced by outliers.
- 2.
- Create a subset of the data by randomly selecting $N(\text{hyperparameter})$ samples from the dataset (with replacement)
 - Use linear regression on the subset to predict a line. Store the equation for this line
 - Repeat the first two steps $M(\text{hyperparameter})$ times
 - Pick a sample to test
 - For each equation you've stored, calculate the output for the sample, then average the outputs. This is the prediction for that sample
 - Do this for all test samples

Since bootstrapping chooses samples at random, the model might not be identical but it will be close. Linear Regression already has low variance because it averages the data and so it won't overfit on outliers. Bagging will still decrease the variance by smoothing out these outliers, but, since the variance is already low, it won't decrease by much. The bias should remain unchanged because bagging doesn't have a strong effect on the bias.

3. Generalized Gradient Boosting Algorithm

Initialize $\forall i F_0(x_i) = 0$

For $m = 1$ to M :

Compute $y'_i = -\frac{\partial L}{\partial F} \Big|_{F_{m-1}(x)}$

Using the dataset and residuals (x, y'_i) , train $h_m(x)$ to predict y'_i

$F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$

To use each loss function in this algorithm, all we have to do is calculate the gradient and plug it into $\frac{\partial L}{\partial F}$

Logistic Loss

$$\begin{aligned}\frac{\partial L}{\partial F} &= \frac{\partial}{\partial F} \log(1 + e^{-yF(x)}) \\ &= \frac{-ye^{-yF(x)}}{1 + e^{-yF(x)}} \\ &= \frac{-ye^{-yF(x)}}{1 + e^{-yF(x)}} \cdot \frac{e^{yF(x)}}{e^{yF(x)}} \\ &= \frac{-y}{1 + e^{yF(x)}}\end{aligned}$$

Hinge Loss

$$\begin{aligned}\frac{\partial L}{\partial F} &= \frac{\partial}{\partial F} \max(0, 1 - yF(x)) \\ &= -y \cdot \text{sign}(1 - yF(x))\end{aligned}$$

4. To derive the recursive expressions, we find the partial derivatives of the loss function and the activation function. Then we can plug them into the backpropagation equations.

Initial Gradient

$$\delta^L = \left[\frac{\partial L}{\partial a^L} \right] \left[\frac{\partial a^L}{\partial z^L} \right]$$

Logistic Loss Partial

$$\begin{aligned}\frac{\partial L}{\partial a^L} &= \frac{\partial}{\partial a^L} \log(1 + e^{-y \cdot a^L}) \\ &= \frac{-y}{1 + e^{y \cdot a^L}}\end{aligned}$$

ReLU Activation Partial

$$\begin{aligned}\frac{\partial a^L}{\partial z^L} &= \frac{\partial}{\partial z^L} \max(0, z^L) \\ &= \begin{cases} 1 & z^L > 0 \\ 0 & z^L \leq 0 \end{cases} \\ &= \text{sign}(z^L)\end{aligned}$$

Back to the Initial Gradient

$$\begin{aligned}\delta^L &= \left[\frac{\partial L}{\partial a^L} \right] \left[\frac{\partial a^L}{\partial z^L} \right] \\ &= \left[\frac{-y}{1 + e^{y \cdot a^L}} \right]^T [\text{diag}(\text{sign}(z^L))]\end{aligned}$$

Now that we have our initial gradient, we can derive recursive expressions for the rest of the gradients.

The Rest of the Gradients

$$\begin{aligned}
\delta^{l-1} &= \delta^l \left[\frac{\partial z^l}{\partial a^{l-1}} \right] \left[\frac{\partial a^{l-1}}{\partial z^{l-1}} \right] \\
&= \delta^l \left[\frac{\partial}{\partial a^{l-1}} (W^l a^{l-1} + b^l) \right] \left[\frac{\partial a^{l-1}}{\partial z^{l-1}} \right] \\
&= \delta^l [W^l] \left[\frac{\partial a^{l-1}}{\partial z^{l-1}} \right] \\
&= \delta^l [W^l] \left[\frac{\partial}{\partial z^{l-1}} \max(0, z^{l-1}) \right] \\
&= \delta^l [W^l] [\text{diag}(\text{sign}(z^{l-1}))]
\end{aligned}$$

Backpropagation Now we can perform the full backpropagation algorithm with these steps:

- (a) Compute all inputs and outputs for each layer (forward pass)
- (b) Compute δ^L for the output layer

$$\delta^L = \left[\frac{-y}{1 + e^{y \cdot a^L}} \right]^T [\text{diag}(\text{sign}(z^L))]$$

- (c) Starting from $l = L - 1$ and working backwards, compute (backward pass)

$$\delta^{l-1} = \delta^l [W^l] [\text{diag}(\text{sign}(z^{l-1}))]$$

- (d) Perform gradient descent

$$b_j^l = b_j^l - \gamma \cdot \delta_j^l$$

$$W_{jk}^l = w_{jk}^l - \gamma \cdot \delta_j^l a_k^{l-1}$$